

BEYOND ASSERTIONS



How to get strong guarantees at compile time

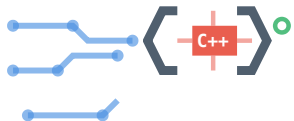
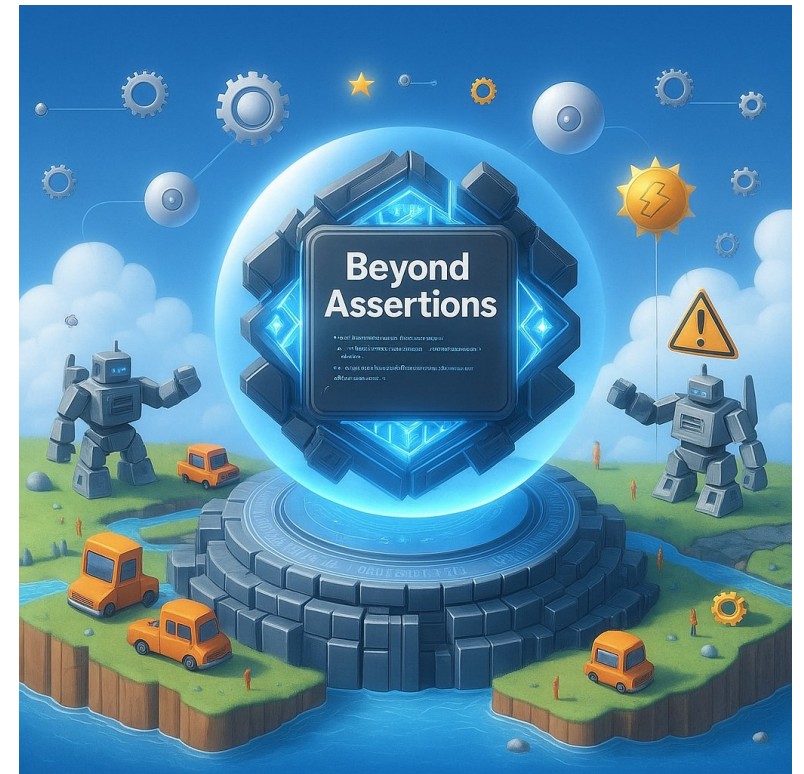
Massimiliano Pagani

maxpag@gmail.com

www.maxpagani.org

September 17, 2024

Italian C++ Community Meetup





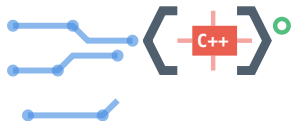
Who's This Guy?

A grumpy old programmer that worked in C++ for 30+ years

Exposed to (BASIC) C, Java, PHP, C#, bash, Scala

Main focus embedded system.

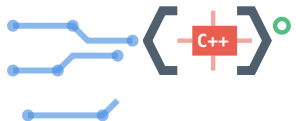
Worked in Video Games, Multimedia, Home/Building Automation, Data Collection, now Flight Software.



Agenda



- Motivation for the Talk
- Assertion and Contracts
- Compile Time Checks
- The case for Refined Types
- Two Examples and GSL comparison
- Limitations
- Conclusions





Motivations for this Talk

- (this is my first public talk, why expose myself?)
- Some time ago...

Finite State Machines (FSM) in Embedded Systems (Part 1) - There's a State in This Machine!



Massimiliano Pagani • February 5, 2024 • 4 comments • 5300 pageviews

Software Development

Tutorials

Algorithms

An introduction to state machines and their implementation. Working from an intuitive definition of

C++ Assertion? Well Yes, But Actually No.



Massimiliano Pagani • April 8, 2024 • 1083 pageviews

Software Development

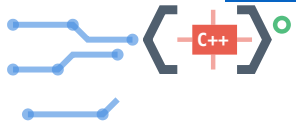
Debugging

C Programming

Assertions are a double-edged sword - on one side you enforce program correctness catching bugs



Assertions

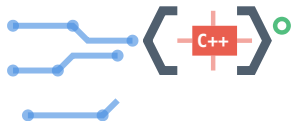




Assertion

- Assertions – (invented in 1947) to **check your expectations**. If my expectations are not met the code won't provide a good result (or behavior)
- Example:

```
float sqrt(float x)
{
    assert( x ≥ 0.0f );
    // complex but uninteresting code to compute the
    // square root of 'x' and placing the result into
    // 'result'
    return result;
}
```





Assertion the details

- Assertions are macros (since K&R 1972)
- Failed condition causes program to abort
- They can be disabled defining NDEBUG and **disabled** also means that **side effects are not executed**.





Assertion problem #1

- Is it ok to disable in release builds?
- NASA wisdom *Fly what you test and test what you fly*
- Time sensitive code could change behavior with and without assertions.
- Side effects are stripped away, consider

```
assert( loadFirmware() == LOAD_OK );
```

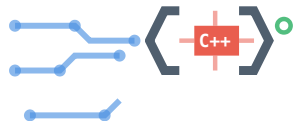




Assertion problem #2

- Failed assertion causes the program to stop
- They are intended for checking internal consistency
- Using them for validation is a misuse, although a common one.
- E.g. I do not expect binary data in a text file, I make sure no binary data enters my code by:

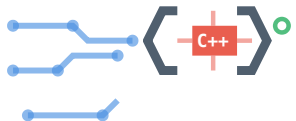
```
assert( std::isctrl(ch) || std::isprint(ch) );
```
- Wrong, cause I have no control on the external file.



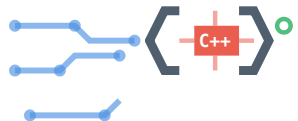


Assertion problem #3

- Assertions catch problems at the **worst possible time**
- When the assertion triggers the problem has already occurred... **elsewhere**
- You can stuff your code with assertion to catch the problem closer to its source, but still you haven't been able to prevent it.



C++ Contracts





What about C++ contracts?



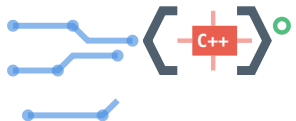
- First language to implement contracts was Eiffel (1986)
- C++26 introduced *contracts* (mere 40 years later)
- Not yet implemented by any major C++ compiler
- Very hard to find information beside the proposal (P2900R14 <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2900r14.pdf>)





What are contracts?

- Specialized construct to enforce function interface requirements
- Preconditions – checked before the function is executed
- Postconditions – checked after the function is executed (before returning to the caller)
- Contract violation causes termination

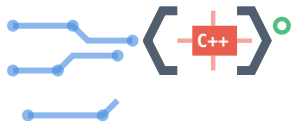


Look of a C++ contract



- Example of precondition:

```
float sqrt(float x)
    pre ( x ≥ 0.0f )
{
    // complex but uninteresting code to compute the
    // square root of 'x' and placing the result into
    // 'result'
    return result;
}
```

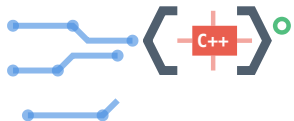


Look of a C++ contract



- Example of postcondition:

```
float sqrt(float const x)
  pre ( x ≥ 0.0f )
  post ( r: r = x*x )
{
    // complex but uninteresting code to compute the
    // square root of 'x' and placing the result into
    // 'result'
    return result;
}
```



C++ contract features 1/2



- 4 types of user defined evaluations:

- Ignore
- Observe
- Enforce
- Quick Enforce

Checking semantic

Call Handler

- Selection of evaluation is implementation defined, but doesn't need to be global



C++ contract features 2/2



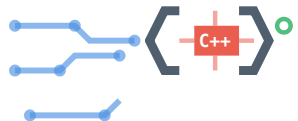
- There is an implementation defined default handler (`::handle_contract_violation()`).
- It is implementation defined whether you can override it or not.
- There are zillions of details (exceptions, lambdas, constexpr, concepts, virtual functions... just to name a few)



Do contracts solve assertion problems?



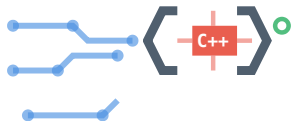
- P#1 (disable) – can be disabled – not solved!
- P#2 (termination) – you may configure the reaction to a failed assertion – not solved (and misuse can't be solved)
- P#3 (run-time check) – contracts are still enforced at run-time – **not** solved!



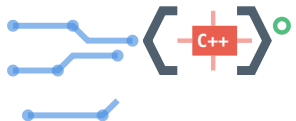
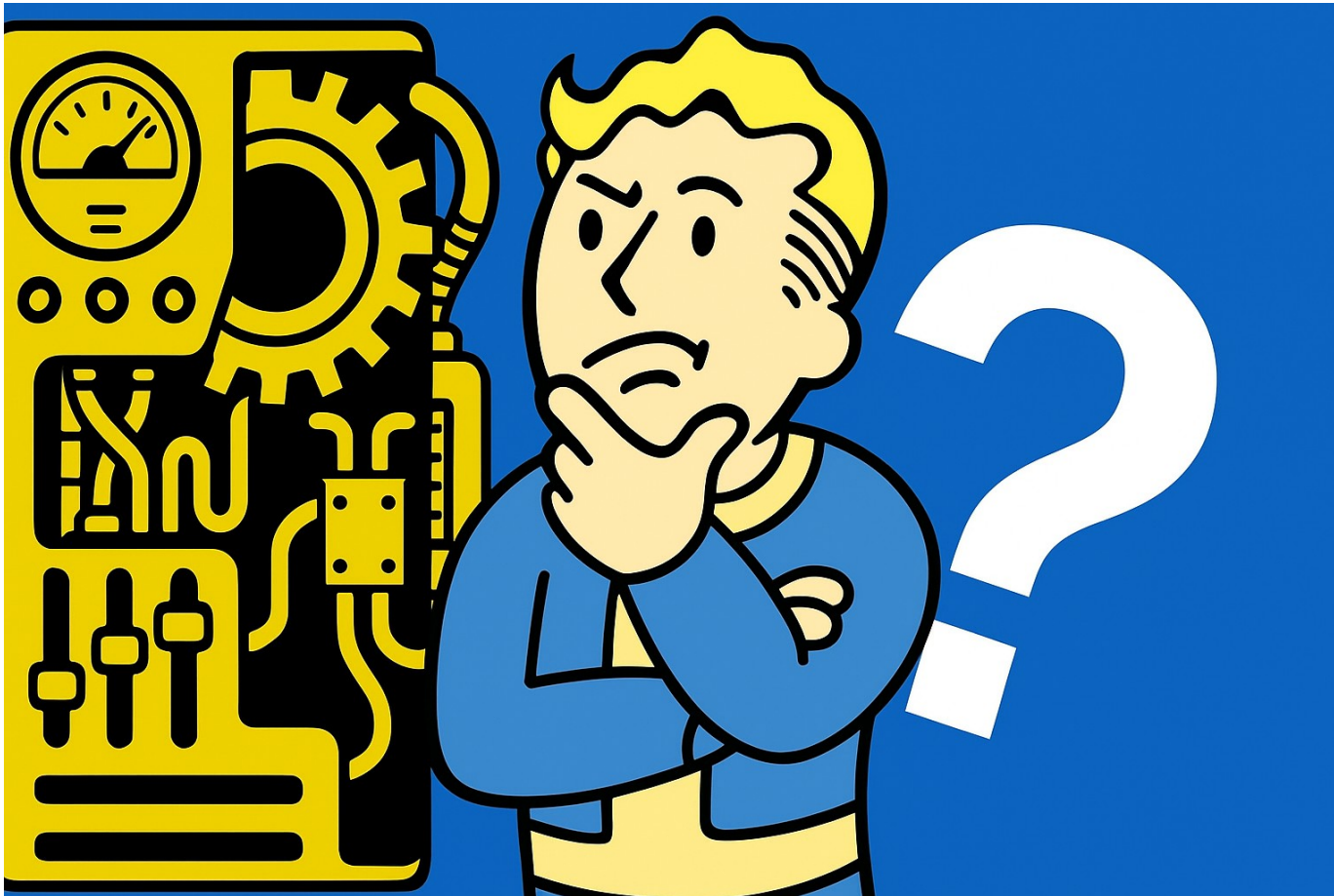
Contracts - conclusions



- Contracts are modern assertions -
 - Better defined
 - Customizable in their outcome
 - Integrated into the language
- But... contracts are still checked at the worse possible time.
- Contracts and assertions are a **safety net** to prevent bugs turning into catastrophes, not helpful to create correct software from the beginning.



Alternatives?





Excursus

- I spent about 4 years shoulder to shoulder with functional programmers
- Odd people
- But with plenty of programming ideas
- One of the most thought provoking was (not the M-word, but):
- **Make Invalid State Unrepresentable**





Excursus

- If the program state can never be invalid, I don't have to check it for being correct.
- This is not different from RAI, we C++ people all use and love.
- If an object exists then it is correct, right?! (I'm looking at you `fstream`).
- No need for an *isValid()* method.





Excursus

- Another point always made by my FP friends was about types. **Rely on Types.**
- Types are checked at compile time and prevent something in the wrong form to reach your code – you can't pass a string to a function that wants an integer...



Excursus



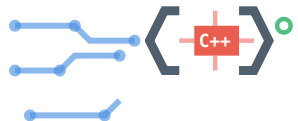
- Wait! Doesn't that look like something you would check with an assertion?
- Enter...



Refined (aka Smart) Types



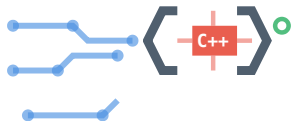
- Types that encode properties
- e.g. an email address can be stored into a string, but it has **additional constraints**. Not all strings are email addresses.
- What about an EmailAddress type that represents only valid email addresses?
- If you get one of these you **don't need to check for validity** before each use – the type system grants this.



Refined (aka Smart) Types



- The constructor must **validate** the arguments.
- You can use constructor+exception or static factory method+expected/optional.
- Manipulation functions must ensure that **only valid entities are produced** (immutability helps).
- A const getter may extract the underlying type for compatibility



How this compare to assertion



- P#1 (disable) – can't be disabled – solved!
- P#2 (termination) – does not apply – solved!
- P#3 (run-time check) – checked at compile time – solved!

If the code compiles, your constraints are satisfied /
You can't compile code that doesn't comply with
contracts.

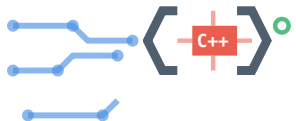


The email address type



```
class EmailAddress
{
    std::string mEmail;
public:
    static bool isAnEmail(std::string_view email) {...}
```

Some
regex
magic





The email address type

```
class EmailAddress
{
    std::string mEmail;
public:
    static bool isAnEmail(std::string_view email) {...}

    explicit EmailAddress(std::string_view email)
    : mEmail(email)
    {
        if( !isAnEmail(email) )
            throw std::invalid_argument("not an email address");
    }
}
```

Very important
to use **explicit**

C++

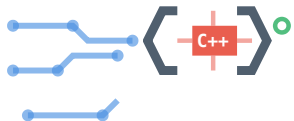
The email address type



```
class EmailAddress
{
    std::string mEmail;
public:
    static bool isAnEmail(std::string_view email) {...}

    explicit EmailAddress(std::string_view email)
    : mEmail(email)
    {
        if( !isAnEmail(email) )
            throw std::invalid_argument("not an email address");
    }

    std::string_view asString() const { return mEmail; };
};
```



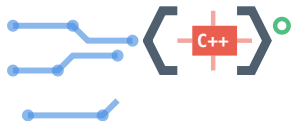
The email address type noexcept (almost)



```
class EmailAddress
{
    std::string mEmail;
public:
    static bool isAnEmail(std::string_view email) {...}

    static std::optional<EmailAddress>
    make( std::string_view email ) {
        return isAnEmail( email ) ? EmailAddress{email} : std::nullopt;
    }

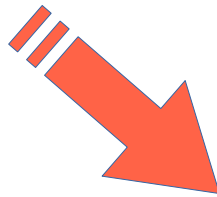
    std::string_view asString() const { return email; };
private:
    explicit EmailAddress(std::string_view email)
        : email(mEmail) {}
};
```



Using Smart Type



```
int processEmail(  
    std::string_view email  
);
```



```
int processEmail(  
    EmailAddress const& email  
);
```





Using Smart Type

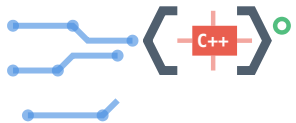
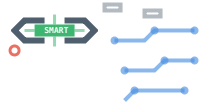
```
processInput( std::string_view recipient )
{
    std::optional<EmailAddress> email{
        EmailAddress::make(recipient);
    }
    if( email.has_value() ) {
        processEmail( email.value() );
    }
    else {
        // error, recipient was not a valid email address
    }
}

processEmail( EmailAddress const& recipient )
{
    // no need for checking, the type ensures only valid
    // email addresses here.
}
```



What next?

- With great powers...
- ... let's solve something important



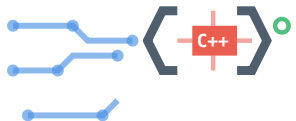
1 BILLION
DOLLAR

MISTAKE



1 BILLION
DOLLAR

MISTAKE



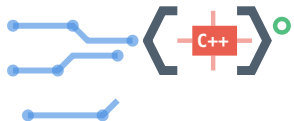
The 1bn \$ Mistake



This is how it has been named the **invention of the NULL pointer** by... Tony Hoare - the inventor of NULL pointer (1965).

(<https://www.infoq.com/presentations/Null-References-The-Billion-Dollar-Mistake-Tony-Hoare/>)

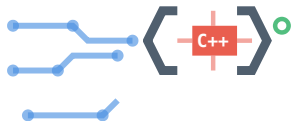
I won't go in depth on why it is wrong, but we all agree on how **life would be better if pointers were not null** when we need them to be valid.



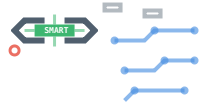


Wait, what about GSL?

- The *mistake* has been recognized by the C++ Core Guideline library
- They designed a `gsl::not_null<T*>` pointer that cannot be assigned to `nullptr`.
- Let's check it out



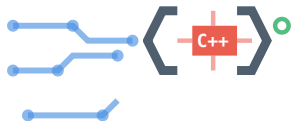
GSL::not_null<T>



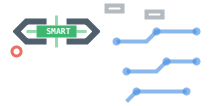
```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    f( nullptr );
    return 0;
}
```



GSL::not_null<T>



```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    f( nullptr );
    return 0;
}
```

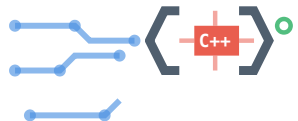
```
$ g++ -Wall -Wextra foo.cc
foo.cc: In function 'int main()':
foo.cc:7:6: error: use of deleted function
'gsl::not_null<T>::not_null(std::nullptr_t) [with T = int*;
std::nullptr_t = std::nullptr_t]'
```

```
    7 |     f( nullptr );
      |     ~^~~~~~
```

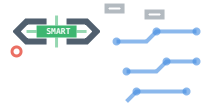
```
In file included from foo.cc:1:
```

```
GSL/include/gsl/pointers:136:5: note: declared here
```

```
   136 |     not_null(std::nullptr_t) = delete;
       |     ^~~~~~
```



GSL::not_null<T>



```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    f( nullptr );
    return 0;
}
```

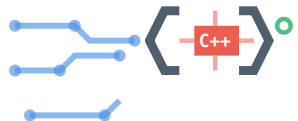
```
$ g++ -Wall -Wextra foo.cc
foo.cc: In function 'int main()':
foo.cc:7:6: error: use of deleted function
'gsl::not_null<T>::not_null(std::nullptr_t) [with T = int*;
std::nullptr_t = std::nullptr_t]'
```

```
    7 |     f( nullptr );
      |     ~^~~~~~
```

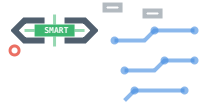
```
In file included from foo.cc:1:
```

```
GSL/include/gsl/pointers:136:5: note: declared here
```

```
136 |     not_null(std::nullptr_t) = delete;
    |     ^~~~~~
```



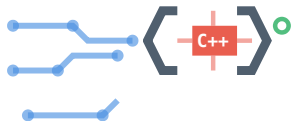
GSL::not_null<T>



```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    int* n = nullptr;
    f( n );
    return 0;
}
```



GSL::not_null<T>

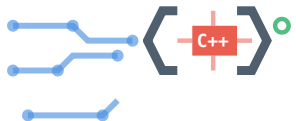


```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    int* n = nullptr;
    f( n );
    return 0;
}
```

```
$ g++ -Wall -Wextra bar.cc
$
```



GSL::not_null<T>

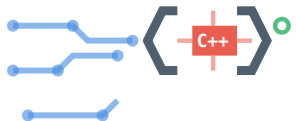


```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    int* n = nullptr;
    f( n );
    return 0;
}
```

```
$ g++ -Wall -Wextra bar.cc
$
```



GSL::not_null<T>



```
#include <gsl/pointers>
```

```
void f( gsl::not_null<int*> p ) { *p = 3; }
```

```
int main()
{
    int* n = nullptr;
    f( n );
    return 0;
}
```

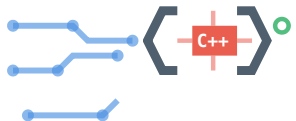
```
$ g++ -Wall -Wextra bar.cc
```

```
$ ./a.out
```

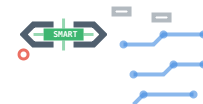
```
terminate called without an active exception
```

```
[1] 650300 IOT instruction (core dumped) ./a.out
```

```
$
```



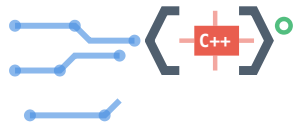
GSL not_null conclusion



- Compile time check for literal assignments or initializations – good
- Run time checks for non literal assignments – bad
- It is a good shot in the right way, but eventually it is **assertion in disguise** for all practical uses.



Roll your own solution!

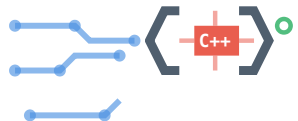


Design your own safe pointer



- The core is not very different from the email example -

```
template<typename T>
class Ptr<T>
{
    T* mPtr;
    explicit Ptr( T* p ) : mPtr( p ) {}
public:
    static std::optional<T> make( T* p ) noexcept {
        return p != nullptr ? Ptr{p} : std::nullopt;
    }
};
```



Design your own safe pointer

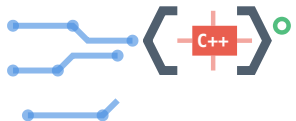


- Add dereference and sum -

```
// ...  
T& operator*() noexcept {  
    return *mPtr;  
}
```

```
std::optional<Ptr<T>>  
Ptr<T>::operator+( ptrdiff_t d ) const noexcept  
{  
    return make( mP + d );  
}
```

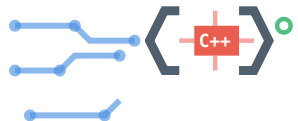
```
// why return optional?  
// b/c ptr+(-(uintptr_t)ptr)/sizeof(*ptr) = 0  
// ...
```



Design your own safe pointer



- Other operators follow
- Some (like +, -) need re-building of the Ptr object and return an optional.
- A few (++ , →) do not need any check
- += and -= cannot be implemented since there is no guarantee that the result is not nullptr.
- (complete implementation here:
<https://gitlab.com/libchef/chef-types>)

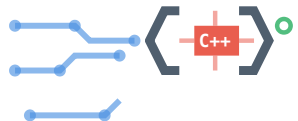


So, are smart types always good?

Yes, but there is a limitation: **Static Type checking works with static properties**. So it won't work with

- Dynamic properties – e.g. I can't restrict a type to objects with a given state
- Dynamic relations – e.g. I can't state that `sqrt(n)` ensures that the result squared equals to `n`.

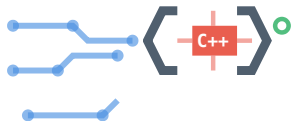
Smart types work best with **immutable** values





Limitations

- C++ allows you to break the rules
- `memset( &t, 0, sizeof(t) )` will throw a wrench in the gears if you rely on type checking to prevent specific values.
- Also **radiations** may twiddle bits in your memory leading to unwanted values.
- But you **can** add assertions inside smart type before dereferencing.





Conclusions

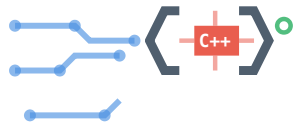
- Assertion implements design by contracts
- DbC enforces contracts at the **worst possible time**, at the point either you ignore the problem or terminate the program
- A viable alternative is to define types so that **invalid states cannot be represented**
- With type checking, contracts are **enforced at compile time** – granting that contracts will always be satisfied
- This approach can be applied almost everywhere, but for dynamic properties.



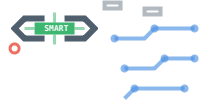
std::end(talk)



- Questions?



std::end(talk)



- Thanks for your time
- Contacts
 - maxpag@gmail.com
 - <https://www.maxpagani.org/>
 - <https://www.linkedin.com/in/maxpagani/>
 -

